ABDK Audit Report: Client Response

Report: Smart Account, ABDK Consulting (Draft)
Contract: `TSmartAccount7702.sol`
Contract version: 0.3.0
Auditors: Dmitry Khovratovich, Mikhail Vladimirov
Client response date: 2026-03-23

A full diff of all changes between v0.3.0 and v1.0.0, covering behavioral changes, security improvements, and the `supportsInterface` bug fix, is documented in `doc/DIFFv0.3.0_1.0.0.md`.

Summary

ID	Severity	Category	ABDK Status	Client Resolution
CVF-1	Major	flaw	Open	Fixed: `9179aaf`, `b99881d`
CVF-2	Moderate	flaw	Open	Fixed: `84c92fa`
CVF-3	Moderate	documentation	Open	Fixed: `8a2573c`
CVF-4	Moderate	behavior	Open	No change; trade-off documented
CVF-5	Moderate	readability	Open	Rejected: would harm ecosystem integration
CVF-6	Minor	procedural	Open	Fixed: `f4fa540`
CVF-7	Minor	procedural	Open	Acknowledged: out of scope
CVF-8	Minor	efficiency	Open	Fixed: `1c3521c`
CVF-9	Minor	efficiency	Open	Obsolete: entire storage system removed
CVF-10	Minor	documentation	Open	Fixed: `432a390`
CVF-11	Minor	typing	Open	Fixed: `0f863e6`

ID	Severity	Category	ABDK Status	Client Resolution
CVF-12	Minor	typing	Open	Fixed: `5d7232f`, `8eefd50`, `7048e0c`

Fixed: **8 / 12**. Not implemented: **1**. Rejected / acknowledged: **3**.

5.1 Major

CVF-1: Storage Collision via OZ `Initializable`

ABDK finding: All contracts based on OpenZeppelin's `Initializable` use the same storage slot for initialization status. Using such contracts for EIP-7702 delegation could lead to storage collisions.

ABDK recommendation: Use unique storage slots for all purposes, including initialization status.

Client analysis: Partially agree: the finding is valid but imprecisely described. OZ v5 `Initializable` already uses ERC-7201 namespaced storage (not a raw low slot as in v4). The actual risk is that all OZ v5 `Initializable` contracts share the *same* ERC-7201 slot for `_initialized`. An EOA that previously delegated to any other OZ `Initializable` contract would find its `_initialized` flag already set, permanently blocking `initialize()` on a fresh delegation to `TSmartAccount7702`. This is a genuine risk for EOAs with delegation history.

Fix applied (initial): Removed `Initializable` inheritance entirely. The `bool initialized` flag was co-located into `EntryPointStorage`, the wallet's own private ERC-7201 namespace (`smartaccount7702.entrypoint`). This guaranteed no collision with any other contract. An `AlreadyInitialized()` custom error was added.

Fix applied (final): The `EntryPointStorage` / `initialize()` approach was itself superseded. While the front-running attack was not possible (`initialize()` required `msg.sender == address(this)`, so only the EOA itself could call it), the architecture had two practical constraints:

1. **Atomic delegation required**: Between the delegation transaction landing and the `initialize()` call landing, the account is delegated but uninitialized; all modifiers revert with `NotInitialized()`. The account is inert during this window, not exploitable, but any UserOp submitted in that gap would fail. To avoid this, delegation and initialization must happen atomically in the same type-4 transaction: the EIP-7702 transaction carries both the authorization tuple and `callData` targeting `initialize(entryPoint)`. Any two-transaction flow (delegate first, initialize second) inherently has a gap.
2. **Sponsorship impossible**: Because only the EOA itself could call `initialize()`, no third party could submit or sponsor the initialization transaction on behalf of the EOA. The EOA had to self-fund its own `initialize()` call, which conflicts with the account-abstraction goal of enabling gasless onboarding via paymasters.

The entire `EntryPointStorage` struct, ERC-7201 namespace, `initialize()` function, `AlreadyInitialized()` error, `EntryPointSet` event, and `_getEntryPointStorage()` helper were removed. The EntryPoint address is now set once, at implementation deployment, via an immutable constructor parameter:

```
address public immutable ENTRY_POINT;

error EntryPointAddressZero();

constructor(address entryPoint_) EIP712("TSmart Account 7702", "1") {
    require(entryPoint_ != address(0), EntryPointAddressZero());
    ENTRY_POINT = entryPoint_;
}
```

The immutable is baked into the implementation bytecode. Every EOA that delegates to this implementation via EIP-7702 shares the same EntryPoint address with no per-EOA initialization step required. Upgrading the EntryPoint requires deploying a new implementation and signing a new authorization tuple, which is the canonical EIP-7702 upgrade path.

Commits: `9179aaf`: `CVF-1: replace OZ Initializable with wallet-local init guard` (superseded) | `6dc652f`: `Remove initialize and use a constant for the entrypoint` | `8d78988`: `Set entrypoint in an immutable variable`

Test results: 127 tests passed, 0 failed after the change.

Suggestion: The immutable approach closes the CVF-1 attack surface permanently. No initialization transaction is required after delegation, which removes an entire class of front-running and misconfiguration risk. The trade-off is that the EntryPoint is fixed per implementation; changing it requires deploying a new implementation contract, which is the intended EIP-7702 upgrade mechanism.

5.2 Moderate

CVF-2: Modifiers Do Not Check Initialization

ABDK finding: The `onlyEntryPoint` and `onlyEntryPointOrSelf` modifiers should also require the account to be already initialized.

ABDK recommendation: Add an initialization check inside the modifiers.

Client analysis: Partially agree. When uninitialized, `entryPoint()` returns `address(0)`. Since `msg.sender` can never equal `address(0)`, the authorization check always reverts; the account is inert, not exploitable. However, reverting with `Unauthorized()` when the root cause is "not yet initialized" is misleading to operators and tooling. An explicit `NotInitialized()` error improves debuggability with no security downside. Severity is closer to Minor than Moderate in practice.

Fix applied (initial): Added `error NotInitialized()`. Both `onlyEntryPoint` and `onlyEntryPointOrSelf` loaded `EntryPointStorage` and reverted with `NotInitialized()` before performing the address comparison.

Fix applied (final): The `NotInitialized` error and the entire `EntryPointStorage` initialization system were removed when the EntryPoint was moved to an immutable constructor parameter (see CVF-1). With an immutable, the EntryPoint is always set, so there is no "uninitialized" state to check. The modifiers now read the immutable directly:

```
modifier onlyEntryPoint() {
    require(msg.sender == ENTRY_POINT, Unauthorized(msg.sender));
    _;
}

modifier onlyEntryPointOrSelf() {
    require(msg.sender == ENTRY_POINT || msg.sender == address(this),
Unauthorized(msg.sender));
    _;
}
```

Note: the modifiers read `ENTRY_POINT` directly rather than calling `entryPoint()`, even though `entryPoint()` is `virtual`. This prevents a subclass override of `entryPoint()` from silently bypassing the authorization check in the modifiers; security-sensitive reads should not go through virtual dispatch.

Commits: `84c92fa`: `CVF-2: revert with NotInitialized on uninitialized protected calls` (superseded) | `6dc652f`: `Remove initialize and use a constant for the entrypoint` | `8d78988`: `Set entrypoint in an immutable variable`

Test results: 127 tests passed, 0 failed after the change.

Suggestion: The immutable approach resolves CVF-2 more completely than the `NotInitialized` check did. The uninitialized state no longer exists, so the entire question of "what should happen before initialization" is eliminated. The `Unauthorized` error now only fires when the wrong caller attempts a protected function, which is its correct and sole purpose.

CVF-3: Inaccurate Comment on `_call`

ABDK finding: The NatSpec comment claims calldata is forwarded "directly without copying to memory", but `calldatacopy` is used, which does copy to memory first.

ABDK recommendation: Fix the comment to match the actual implementation.

Client analysis: Agree: the comment is factually wrong. The intent was to convey that the function avoids a Solidity ABI-encode round-trip, but the phrase "without copying to memory" is incorrect. The EVM `CALL` opcode always reads its input data from memory; `calldatacopy` is mandatory in this assembly path.

Fix applied: Updated the `_call()` NatSpec to accurately describe the mechanism while preserving the optimization intent:

```
/// @dev Executes a call and bubbles up revert data on failure.
///      Uses assembly to copy calldata into memory and forward it via CALL,
///      avoiding a Solidity ABI-encode round-trip for large payloads.
```

Commit: `8a2573c`: `CVF-3-docs: correct _call NatSpec for calldata memory copy`

Suggestion: Documentation-only change; no runtime behavior affected. The optimization note is important to retain, as it explains why assembly is used at all (skipping `abi.encode` allocation), which would otherwise appear as unnecessary complexity to reviewers.

CVF-4: `_call` Drops Return Data on Success

ABDK finding: The function discards data returned by the nested call.

ABDK recommendation: Forward the returned data to the caller.

Client analysis: Disagree for the primary ERC-4337 use case; the trade-off is real and must be documented.

The `execute` function is called by the EntryPoint as part of UserOp execution. The EntryPoint does not use the return value of `execute`; it cannot, because the `IAccount` interface declares `execute` as returning nothing. All major ERC-4337 reference implementations (Alchemy Light Account, Coinbase SmartWallet, Solady SimpleAccount) drop return data from `execute` for the same reason.

However, two considerations deserve attention:

1. **Direct self-call path**: `onlyEntryPointOrSelf` permits `msg.sender == address(this)`. When the EOA calls `execute()` directly (type-4 transaction to self), it is the caller and may legitimately need the return value, for example when reading an output amount from a DEX call. That value is silently lost today.
2. **The return data buffer is still live**: Because `_call` is `internal` (compiled as a JUMP, not a CALL), after `_call` returns to `execute`, `returndatasize()` still holds the nested call's output. Forwarding it would not require re-execution; only `returndatacopy` + assembly `return` in `execute`. The EntryPoint ignores it (backward-compatible). The only real cost is gas proportional to return data size (~30–50 gas for a typical 32-byte return).

Resolution: No logic change. A NatSpec note was added to `execute()` documenting that return data is intentionally discarded in the ERC-4337 path and directing callers that require return values to override `execute` (which is `virtual`) or call the target directly.

Commit: `d5db86b`: `docs: add CVF-4 return-data NatSpec, fix FEEDBACK priority table, update CVF-9 report suggestion`

Suggestion: If the wallet is expected to be used heavily in the direct self-call path (outside the EntryPoint), consider adding a forwarding override in a subclass or a separate `executeWithResult` function. The assembly to forward return data is straightforward and fully backward-compatible with the EntryPoint flow.

CVF-5: Revert Data Not Wrapped in Named Error

ABDK finding: Forwarding raw revert data makes it impossible to reliably distinguish error messages from nested calls from error messages from the contract itself.

ABDK recommendation: Wrap the revert data in a named error.

Client analysis: Disagree: the recommendation would actively harm ecosystem integration and is architecturally unnecessary for this contract.

Three reasons:

1. **Bundlers and simulators**: ERC-4337 bundlers parse revert selectors from `execute` to classify failures. Wrapping changes the selector and breaks this parsing.

2. **Off-chain tooling**: Etherscan, Tenderly, and other debuggers decode revert reasons by selector. Wrapping buries the original error under a new selector, making traces harder to read.
3. **The ambiguity concern is moot here**: The wallet's own errors (`Unauthorized`, `EmptyBytecode`) are thrown exclusively from modifiers and pre-call guards, which execute *before* `_call` is ever reached. By the time `_call` executes, access control has already passed. Any revert that bubbles up from `_call` is always from the nested external call, never from the wallet itself. There is no actual ambiguity to resolve.

The raw bubble-up pattern is the correct standard behavior, consistent with EIP-7821 and all major smart wallet implementations.

Resolution: No change to revert forwarding. A comment was added to `_call` explicitly documenting the intentional bubble-up design and why wrapping is not used.

Commit: None.

Suggestion: If ABDK's concern is about distinguishing wallet-level errors from nested-call errors in tooling, the correct solution is to document that wallet errors are only ever emitted before `_call` executes. Wrapping revert data is not the answer. This is already addressed by the contract structure and the added comment.

5.3 Minor

CVF-6: Exact Compiler Version Pragma

ABDK finding: Specifying `pragma solidity 0.8.34;` makes migration to newer compiler versions harder.

ABDK recommendation: Use `^0.8.0` or `^0.8.34`.

Client analysis: Partially agree. `^0.8.0` is overly permissive for a production smart account: it would allow any 0.8.x compiler and could introduce behavioral differences across patch versions. `^0.8.34` is the correct middle ground: it enforces the minimum version (0.8.34, required for Prague EVM / EIP-7702 support) while allowing patch-level updates from future compiler releases.

Fix applied: Changed `pragma solidity 0.8.34;` to `pragma solidity ^0.8.34;`.

Commit: `f4fa540`: `CVF-6: relax pragma from exact pin to ^0.8.34`

Suggestion: The minimum of 0.8.34 is not arbitrary. It is the first compiler release with Prague EVM support needed for EIP-7702 testing. This constraint should be documented in `foundry.toml` as well (`solc = "0.8.34"` or a minimum version comment) to ensure reproducible builds despite the relaxed pragma.

CVF-7: Dependencies Not Reviewed

ABDK finding: `IAccount` and `PackedUserOperation` from the `account-abstraction` package were not reviewed.

Client analysis: Acknowledged: these interfaces are from the canonical ERC-4337 `account-abstraction` repository (v0.9.0), which has received independent audits from multiple reputable firms. They are minimal interface definitions with no logic. `IAccount` is a single function signature and `PackedUserOperation` is a struct definition. The audit scope gap is noted but no action is required on the client side.

Resolution: Acknowledged; no code change.

Commit: None.

Suggestion: For the final audit report, ABDK could explicitly note that the `account-abstraction` v0.9.0 interfaces are out of scope and reference the upstream audit trail. This avoids implying an unreviewed risk where the actual risk is near-zero.

CVF-8: Custom Errors Without Parameters

ABDK finding: The custom errors `Unauthorized()` and `EmptyBytecode()` would be more useful with parameters.

ABDK recommendation: Add parameters to the custom errors.

Client analysis: Agree: adding an `address caller` parameter to `Unauthorized` marginally improves on-chain observability. In practice, the unauthorized caller is already recoverable from transaction context and call traces, so the improvement is minor. `EmptyBytecode` has no meaningful parameter to add.

Fix applied: Changed `Unauthorized()` to `Unauthorized(address caller)`. All authorization revert sites updated to pass `msg.sender`. `EmptyBytecode()` left unchanged.

```
/// @notice Thrown when the caller is not authorized.
/// @param caller The address that attempted the unauthorized call.
error Unauthorized(address caller);
```

Commit: `1c3521c`: `CVF-8: only unauthorized revert encoding now includes msg.sender`

Suggestion: The `caller` field adds a small overhead to revert data encoding, but this is only paid when the revert branch is taken; on the successful path the cost is zero. The real trade-off is a slight bytecode size increase. Acceptable for the observability benefit.

CVF-9: Hardcoded Hash Value Instead of Expression

ABDK finding: The Solidity compiler can precompute constant hash expressions. Use the expression instead of the hardcoded value.

ABDK recommendation: Replace the hardcoded `ENTRY_POINT_STORAGE_LOCATION` literal with the ERC-7201 derivation expression.

Client analysis: Agree in principle: the expression form is more readable and self-verifying. The comment above the constant already contains the derivation formula, so moving it into the constant declaration would make the connection explicit.

Resolution: Fully obsolete. The `ENTRY_POINT_STORAGE_LOCATION` constant, the `EntryPointStorage` struct, `_getEntryPointStorage()`, and the entire ERC-7201 namespaced storage system were removed when the EntryPoint address was moved to an immutable constructor parameter (see CVF-1). There is no storage slot to reference, no assembly slot assignment, and no `StorageLocation.t.sol` test (deleted). The finding no longer applies.

Commit: None (finding superseded by architectural change).

CVF-10: Empty Blocks Without Inline Comments

ABDK finding: It is good practice to put a comment into an empty block to explain why the block is empty.

Client analysis: Partially agree. NatSpec comments above both functions already document their intent, and the section header explains why token callbacks are needed under EIP-7702. However, an inline comment inside `{}` makes it immediately clear the empty body is intentional, not a forgotten implementation, without requiring the reader to look at the surrounding NatSpec.

Fix applied: Added an `// Intentionally empty:` inline comment to both `receive()` and `fallback()`:

```
receive() external payable {
    // Intentionally empty: ETH accepted unconditionally.
}

fallback() external payable {
    // Intentionally empty: unknown calls accepted to maintain EOA-equivalent
behavior.
}
```

Both bodies must stay empty:

- `receive()`: `transfer`/`send` forward only 2300 gas, leaving no headroom for logic.
- `fallback()`: accepting unknown selectors silently preserves EOA-equivalent behavior; reverting would break integrations that probe the EOA with arbitrary calldata.

Commit: `432a390`: `CVF-10: add intentionally-empty inline comments to receive and fallback`

Suggestion: None. The fix is minimal and complete.

CVF-11: Version Number Should Be a Named Constant

ABDK finding: The hardcoded string `"0.3.0"` returned by `version()` should be a named constant.

Client analysis: Agree: a named constant makes the version string grep-able, prevents divergence between the value returned at runtime and the value referenced in documentation, and makes future version bumps a single-location change.

Fix applied: Added `string private constant VERSION = "0.3.0";`. Updated `version()` to return `VERSION`.

```
string private constant VERSION = "0.3.0";

function version() external pure virtual returns (string memory) {
    return VERSION;
}
```

Commit: `0f863e6`: `CVF-11-feat: replace hardcoded version string with named VERSION constant in TSmartAccount7702`

Suggestion: When the contract is updated, `VERSION` should be bumped as part of the same commit as the functional change, not as a separate cleanup. Consider enforcing this in the PR checklist.

CVF-12: Interface Signatures Should Be Named Constants

ABDK finding: Magic byte literals in `supportsInterface` should be named constants.

Client analysis: Agree, but prefer `type(...).interfaceId` over named `bytes4` constants wherever the Solidity interface type is available. `type(IERC165).interfaceId` is self-verifying: it computes the correct value from the actual interface definition, and any mismatch between the constant and the interface becomes a compile-time impossibility.

Note: `type(IAccount).interfaceId` was already used for the first check. The remaining five checks used raw `bytes4(0x...)` literals.

Fix applied: Added imports for `IERC165`, `IERC1271`, `IERC721Receiver`, `IERC1155Receiver`. Updated `supportsInterface()` to use `type(...).interfaceId` for all standard interfaces. The magic return values in `onERC721Received`, `onERC1155Received`, and `onERC1155BatchReceived` were also replaced with their respective `.selector` expressions (`IERC721Receiver.onERC721Received.selector`, `IERC1155Receiver.onERC1155Received.selector`, `IERC1155Receiver.onERC1155BatchReceived.selector`). ERC-7739 was initially included as a named constant `ERC7739_INTERFACE_ID = 0x77390001` but was subsequently removed (see post-fix note below).

```
function supportsInterface(bytes4 interfaceId) public view virtual returns
(bool) {
    return interfaceId == type(IAccount).interfaceId
        || interfaceId == type(IERC1271).interfaceId
        || interfaceId == type(IERC721Receiver).interfaceId
        || interfaceId == type(IERC1155Receiver).interfaceId
        || interfaceId == type(IERC165).interfaceId;
}
```

Commit: `5d7232f`: `CVF-12: refactor: use named interface ID constants in src supportsInterface and validate exact bytes4 IDs in tests`

Test note: During this change, tests confirmed the `IAccount` interface ID is `0x19822f7c` (not `0x3a871cdd` as previously hardcoded). The constant was corrected. This validates that using `type(...).interfaceId` catches exactly this class of silent mismatch.

Post-fix correction: The initial fix included `ERC7739_INTERFACE_ID = 0x77390001` in `supportsInterface`. This was removed after further analysis: ERC-7739 defines no new function signatures and therefore has no ERC-165 interface ID. The `0x77390001` value is a detection sentinel returned by `isValidSignature(0x7739...7739, "")`, not a computed ERC-165 interface ID. Advertising it via `supportsInterface` was non-standard and has been removed. The `bytes4 private constant ERC7739_INTERFACE_ID` declaration was also removed. ERC-7739 detection must use the `isValidSignature` path as specified by the standard. Commit: `8eefd50`.

The same principle was extended to the token receiver callbacks: the hardcoded magic literals in `onERC721Received`, `onERC1155Received`, and `onERC1155BatchReceived` were replaced with their respective `.selector` expressions. Direct return value tests were added using the spec-mandated hardcoded constants as independent verification. Commit: `7048e0c`.

Suggestion: `IERC1155Receiver` inherits from `IERC165`. `type(IERC1155Receiver).interfaceId` does NOT include inherited interface selectors per the ERC-165 spec; it covers only the functions declared directly in `IERC1155Receiver`. This is the correct behavior for the `supportsInterface` check. The explicit `type(IERC165).interfaceId` entry in the return expression covers the inherited base separately, which is the right pattern.

Open Items

The following findings have no associated code change and remain as documented trade-offs or acknowledged scope gaps:

ID	Resolution	Rationale
CVF-4	No change; trade-off documented in NatSpec	Return data dropped intentionally for EntryPoint compatibility; `execute` is `virtual` for override
CVF-5	Rejected	Wrapping revert data would break ERC-4337 bundlers, off-chain tooling, and EIP-7821 compatibility
CVF-7	Acknowledged	`IAccount` / `PackedUserOperation` interfaces are from audited upstream (account-abstraction v0.9.0) with no logic
CVF-9	Obsolete	Entire ERC-7201 storage system removed; `ENTRY_POINT_STORAGE_LOCATION` no longer exists

---

Supplementary Fixes (Post-Audit, Independent Review)

The following changes were identified during an independent internal code review after the ABDK audit. They are not part of the ABDK report.

CR-1: Zero-Address Guard for EntryPoint

Description: When the EntryPoint was set via `initialize()`, passing `address(0)` would permanently brick the account. `onlyEntryPoint` always reverted with `Unauthorized` since `msg.sender` can never equal `address(0)`, with no in-contract remedy. An initial fix added `AddressZeroForEntryPointNotAllowed()` in `initialize()`.

With the move to an immutable constructor parameter (see CVF-1), the guard was relocated to the constructor with a dedicated custom error:

```
error EntryPointAddressZero();

constructor(address entryPoint_) EIP712("TSmart Account 7702", "1") {
    require(entryPoint_ != address(0), EntryPointAddressZero());
    ENTRY_POINT = entryPoint_;
}
```

Deploying with `address(0)` now reverts at construction time with `EntryPointAddressZero()`, so no bricked implementation can ever exist. The previous `AddressZeroForEntryPointNotAllowed` error name was replaced with the shorter `EntryPointAddressZero`.

Commits: `ef3fd38`: initial guard in `initialize()` (superseded) | `6dc652f`: `Remove initialize and use a constant for the entrypoint` | `8d78988`: `Set entrypoint in an immutable variable`

CR-6: Signature Checked Before Prefund Payment

Description: `validateUserOp()` previously paid the ETH prefund to the EntryPoint before validating the signature. `_rawSignatureValidation` (ecrecover-based, from `SignerEIP7702`) never reverts; it only returns `true` or `false`. On `SIG_VALIDATION_FAILED`, the prefund had already been transferred and was retained by the EntryPoint to cover the bundler's gas cost. Checking the signature first avoids this unnecessary ETH loss.

The ERC-4337 protocol places the responsibility for not submitting failing UserOps on the bundler and EntryPoint (via off-chain simulation), not on the account. The account should still minimise ETH exposure in the validation path.

Previous version: [TSmartAccount7702.sol#L134]

```
 if (missingAccountFunds > 0) {
            assembly ("memory-safe") {
                pop(call(gas(), caller(), missingAccountFunds, 0x00, 0x00,
0x00, 0x00))
            }
        }

if (!_rawSignatureValidation(userOpHash, userOp.signature)) {
            return 1;
}
```

New version

```
 if (!_rawSignatureValidation(userOpHash, userOp.signature)) {
      return 1;
}

if (missingAccountFunds > 0) {
         assembly ("memory-safe") {
                pop(call(gas(), caller(), missingAccountFunds, 0x00, 0x00,
0x00, 0x00))
             }
}
```

Fix applied: Moved `_rawSignatureValidation` before the `missingAccountFunds` transfer. Invalid-signature UserOps no longer cause any ETH to leave the account.

Commit: `920de4c`: `check signature before paying prefund in validateUserOp`

Style: `require(condition, CustomError())` Guards

Description: All `if (!condition) revert CustomError()` guards were replaced with the `require(condition, CustomError())` form introduced in Solidity 0.8.26. The two styles produce identical bytecode; this is a purely stylistic change for consistency and readability, aligned with the OpenZeppelin v5 convention already used by the contract's dependencies.

The one exception is `if (!_rawSignatureValidation(...)) return 1` in `validateUserOp`, which was intentionally left as `if`; it returns a value rather than reverting and cannot be expressed as a `require`.

Commit: `82b698c`: `refactor: replace if/revert guards with require(condition, CustomError()) style`

ERC-7739: Removed from `supportsInterface`

Description: The initial CVF-12 fix included `ERC7739_INTERFACE_ID = 0x77390001` in `supportsInterface`. After further analysis, ERC-7739 defines no new function signatures and therefore has no ERC-165 interface ID. The `0x77390001` value is the detection sentinel returned by `isValidSignature(0x7739...7739, "")`, not a computed ERC-165 interface ID (XOR of function selectors). Advertising it via `supportsInterface` was non-standard.

Fix applied: Removed the `ERC7739_INTERFACE_ID` constant and its entry from `supportsInterface`. Added a NatSpec comment explaining that ERC-7739 detection must use the `isValidSignature` path as specified by the standard.

Commit: `8eefd50`: `fix: remove ERC-7739 from supportsInterface — no ERC-165 interface ID exists, detection is via isValidSignature`

Token Receiver Callbacks: Selector Expressions

Description: The magic return values in `onERC721Received`, `onERC1155Received`, and `onERC1155BatchReceived` were hardcoded `bytes4` literals. This is the same class of issue as CVF-12: a literal can silently diverge from the spec if copy-pasted incorrectly.

Fix applied: Replaced literals with `.selector` expressions (`IERC721Receiver.onERC721Received.selector`, `IERC1155Receiver.onERC1155Received.selector`, `IERC1155Receiver.onERC1155BatchReceived.selector`). The spec-mandated values (`0x150b7a02`, `0xf23a6e61`, `0xbc197c81`) were verified against `erc-721.md` and `erc-1155.md`. Direct return value tests were added using hardcoded spec constants as independent verification.

Commit: `7048e0c`: `refactor: replace magic bytes4 literals in token receiver callbacks with selector expressions`

Immutable EntryPoint: Elimination of Initialization Attack Surface

Description: The `EntryPointStorage` / `initialize()` architecture introduced in response to CVF-1 was secure against front-running: `initialize()` required `msg.sender == address(this)`, meaning only the EOA itself could call it. A third party, including an attacker, could not invoke `initialize()` on someone else's account. So the front-running attack was not possible.

However, this same access control created two constraints:

1. **Atomic delegation required**: Between the delegation transaction landing and the `initialize()` call landing, the account is delegated but uninitialized: inert but not exploitable. To avoid any gap, delegation and initialization must happen in the same type-4 transaction (a type-4 transaction can carry both the delegation authorization tuple and a `callData` field, so the EOA can embed `initialize(entryPoint)` in its own delegation transaction). Any two-transaction flow leaves a window where submitted UserOps would fail with `NotInitialized()`.
2. **Sponsorship impossible**: Because only the EOA could call `initialize()`, no third party (relayer, paymaster, bundler) could submit or pay for the initialization. The EOA had to self-fund its own `initialize()` call, which contradicts the account-abstraction goal of gasless onboarding. The atomic type-4 wrapper pattern would still require the EOA to pay gas for the delegation transaction itself.

The immutable approach eliminates the problem entirely: no initialization transaction is needed at all, so there is nothing to sponsor, sequence, or coordinate.

Fix applied: The entire initialization system was removed. The EntryPoint is now baked into the implementation bytecode as an immutable set at construction time:

```
address public immutable ENTRY_POINT;

constructor(address entryPoint_) EIP712("TSmart Account 7702", "1") {
    require(entryPoint_ != address(0));
    ENTRY_POINT = entryPoint_;
}
```

All EOAs that delegate to the same implementation share the same `ENTRY_POINT` value; there is no per-EOA state to initialize, race, or misconfigure. Upgrading to a different EntryPoint requires deploying a new implementation contract and signing a new EIP-7702 authorization tuple, which is the canonical upgrade path for EIP-7702 accounts.

Removed artifacts: `EntryPointStorage` struct, `ENTRY_POINT_STORAGE_LOCATION` constant, `_getEntryPointStorage()` helper, `initialize()` function, `AlreadyInitialized()` error, `NotInitialized()` error, `AddressZeroForEntryPointNotAllowed()` error (replaced by `EntryPointAddressZero()`), `EntryPointSet` event, `StorageLocation.t.sol` test file, all per-EOA initialization calls across test suites, and five initialization-specific attack tests in `AttackTests.t.sol`.

A `TSmartAccount7702V09` mock (`test/mocks/TSmartAccount7702V09.sol`) was added for tests targeting the EntryPoint v0.9.0 canonical address (`0x433709009B8330FDa32311DF1C2AFA402eD8D009`):

```
contract TSmartAccount7702V09 is TSmartAccount7702 {
    constructor() TSmartAccount7702(0x433709009B8330FDa32311DF1C2AFA402eD8D009)
{}
}
```

Commits: `6dc652f`: `Remove initialize and use a constant for the entrypoint` | `8d78988`: `Set entrypoint in an immutable variable`

Test results: 127 tests passed, 0 failed.